ATC Timetable Management System

models/manualTimetableModel.js — Refactor Report

9 findings, each with the current code, why it matters, and the fix direction — ordered from a real correctness bug down to readability cleanups.

Codebase: refactoredCodes | Date: 20 August 2026

#	Finding	Severity
1	Transaction runs on the shared pool, not one dedicated connection	CRITICAL
2	resolveSlotColumn re-queries the DB up to 18 times per request	HIGH
3	SLOT_RANGES / programSlotMatch duplicated across 3 files	MEDIUM
4	programSlotMatch invoked 2–3x for the same inputs in one loop pass	LOW
5	The same hours-remaining rule is checked twice, written two ways	LOW
6	The "1.5 hour" business constant is hardcoded in 3 places in this file	MEDIUM
7	Magic numbers (18, 7, 8) not extracted into named constants	LOW
8	SHOW COLUMNS FROM venues re-run on every single call	MEDIUM
9	Inconsistent failure style: some paths throw, others log-and-return	LOW

1. Transaction runs on the shared pool, not one dedicated connection

CRITICAL — DATA INTEGRITY

db is the shared connection pool (import db from '../db.js', where db.js does mysql.createPool(...)) — not a single dedicated connection. Every .query() call on a pool can be served by a different underlying connection under concurrent load.

```
await db.query("START TRANSACTION");
...
await db.query(`INSERT INTO extracted_timetables (...) VALUES (...)`, [...]);
await db.query(`UPDATE venues SET \`${sInfo.statusCol}\` = 'used' WHERE venue_id = ?`, [venue_id]);
...
await db.query(`UPDATE subjects SET ltpa = COALESCE(ltpa, 0) + ? WHERE subject_id = ?`, [...]);
await db.query("COMMIT");
```

WHY THIS MATTERS

START TRANSACTION might run on connection A from the pool, but the very next INSERT could get routed to connection B — which was never told to start a transaction at all. Under load (two tmasters using Manual Timetable at the same time, or overlapping requests), this transaction's atomicity is not actually reliable. A failure partway through could leave one slot inserted and the other not — exactly the scenario the transaction exists to prevent.

THE PATTERN ALREADY USED CORRECTLY ELSEWHERE IN THIS CODEBASE

```
// logics/timetablesLogic.js :: handleUpdatetimetable
const conn = await pool.getConnection();
await conn.beginTransaction();
...conn.query(...) calls, all on the SAME conn...
await conn.commit();
conn.release();
```

Fix direction: check out one connection at the top of addtimetable via pool.getConnection(), run every query in the function on that same connection object, and release it in a finally block.

2. resolveSlotColumn re-queries the database up to 18 times per request

HIGH — PERFORMANCE

```
if (typeof slot === "string") {
  for (const statusCol of candidates) {          // up to 18 candidates
    const timeCol = statusCol.replace(/_status$/, "");
    const [rows] = await db.query(
      `SELECT \`${timeCol}\` AS timeVal FROM venues WHERE venue_id=? LIMIT 1`, [venue_id]
    );                                             // <-- one query PER candidate
    if (rows.length && rows[0].timeVal.trim() === slot.trim()) {
      return statusCol;
    }
  }
}
```

THE DATA IS ALREADY AVAILABLE ONE CALLER UP

```
// addtimetable() already does this BEFORE calling resolveSlotColumn:
const [venueDataRes] = await db.query(`SELECT * FROM venues WHERE venue_id = ?`, [venue_id]);
const V = venueDataRes[0];   // <-- V already has every monday_slotN column loaded

currentStatusCol = await resolveSlotColumn({ day, slot, venue_id });
//                         ^ venue_id is passed, but the already-fetched V is not
```

Fix direction: change resolveSlotColumn to accept the already-fetched venue row (resolveSlotColumn({ day, slot, venue })) and compare the slot string against that in-memory object's fields directly. This removes up to 18 database round trips down to zero extra queries for this step.

3. SLOT_RANGES / programSlotMatch duplicated across three files

MEDIUM — MAINTAINABILITY

```
const SLOT_RANGES = {
  "full-time": [ "07:30-08:15", ... 12 entries ... ],
  "evening":   [ "16:25-17:10", ... 8 entries ... ],
  "VETA":      [ ... 12 entries, identical to full-time ... ],
  "full-evening": ["16:25-17:10", "17:10-17:55"]
};
const programSlotMatch = (programType, slotTime) => { ... };
```

CONFIRMED ELSEWHERE IN THIS ENGAGEMENT (Business-Process-Reconstruction-Report.docx, BR-01)

The identical object and function exist in models/tmasterModel.js too. A third, INDEPENDENTLY DRIFTED copy exists in logics/collisionMonitorLogic.js, which used underscored values ("full_time"/"veta") that never matched real data ("full-time"/"VETA") — confirmed against the live production database. That drift had already happened once before it was caught.

Fix direction: extract SLOT_RANGES and programSlotMatch into a shared module (e.g. utils/slotRules.js) imported by tmasterModel.js, manualTimetableModel.js, and collisionMonitorLogic.js. This is the highest-leverage change of the three "duplication" findings in this report, since it removes the possibility of the exact drift that already happened once from happening again.

4. programSlotMatch invoked 2–3x for the same inputs, in one loop pass

LOW — READABILITY / MINOR EFFICIENCY

```
// Program Type vs Slot Time compatibility
if (!programSlotMatch(S.program_type, sInfo.slotTime)) {
  reasons.push(`PROGRAM TYPE MISMATCH: "${S.program_type}" not compatible with slot "${sInfo.slotTime}"`);
}

// If any conflict in this slot -> cannot assign
if (tutorConflict || venueConflict || programConflict || !programSlotMatch(S.program_type, sInfo.slotTime)) {
  canAssign = false;
}
```

programSlotMatch(S.program_type, sInfo.slotTime) is a pure function — same inputs, same output, every time — but it's called twice here for the identical pair of arguments (and a third time earlier in Part 6's next-slot check, for the current-slot case specifically).

FIX DIRECTION

```
const typeMismatch = !programSlotMatch(S.program_type, sInfo.slotTime);
if (typeMismatch) { reasons.push(`PROGRAM TYPE MISMATCH...`); }
if (tutorConflict || venueConflict || programConflict || typeMismatch) { canAssign = false; }
```

5. The same hours-remaining rule is checked twice, written two different ways

LOW — READABILITY

```
// Early in the function, per subject:
const remainingHours = totalHours - currentLtpa;
if (remainingHours < 1.5) { ...skip... }

// Later, right before the insert:
const ltpaIncrement = 1.5;
if (currentLtpa + ltpaIncrement > totalHours) { ...skip... }
```

totalHours - currentLtpa < 1.5 and currentLtpa + 1.5 > totalHours are the same inequality, algebraically rearranged. Not wrong, but reads like two different rules to anyone skimming the function. Fix direction: express both checks the same way (or compute remainingHours once and reuse it for both).

6. The "1.5 hour" business constant is hardcoded in three places in this file

MEDIUM — MAINTAINABILITY

```
// getTimetablesFromDB:
WHERE (ltpa + 1.5) <= total_hours_per_week

// getDistinctValues:
WHERE (ltpa + 1.5) <= total_hours_per_week          // identical clause, copy-pasted

// addtimetable:
const ltpaIncrement = 1.5; // Double slot = 1.5 hours
```

All three encode the same "one manual assignment is worth 1.5 hours" rule independently. Fix direction: a single const DOUBLE_SLOT_HOURS = 1.5; at the top of the file, referenced by all three, so the rule only needs to change in one place if it ever does.

7. Magic numbers not extracted into named constants

LOW — READABILITY

```
if (currentSlotNum >= 18) { canAssignDouble = false; ... }              // 18 = ?

const isFriday = dayUpper === "FRIDAY";
if (isFriday && (
  (currentSlotNum >= 7 && currentSlotNum <= 8) ||
  (nextSlotNum >= 7 && nextSlotNum <= 8)
)) { canAssignDouble = false; ... }                                      // 7, 8 = ?
```

Fix direction: const MAX_SLOTS_PER_DAY = 18; and const FRIDAY_BREAK_SLOTS = [7, 8]; (or similar), so the Friday-break and end-of-day rules are self-documenting at the point of use.

8. SHOW COLUMNS FROM venues re-run on every single call

MEDIUM — PERFORMANCE

```
async function resolveSlotColumn({ day, slot, venue_id }) {
  const dayLower = day.toLowerCase();
  const [cols] = await db.query(`SHOW COLUMNS FROM venues`);      // <-- every single call
  const colNames = cols.map(c => c.Field);
  const candidates = colNames.filter(c => c.startsWith(dayLower) && c.endsWith("_status")).sort();
  ...
```

The venues table's schema doesn't change between requests — it's asked fresh every time anyway. Since the day/slot column-naming pattern is fixed and known (`{day}_slot{1-18}` / `{day}_slot{1-18}_status`), this could be generated directly in code instead of discovered via a query — removing another database round trip, on top of the fix in Finding 2.

9. Inconsistent failure style: some paths throw, others log-and-return

LOW — CONSISTENCY

```
// Parameter validation: throws
if (!day || !venue_id || ...) {
  throw new Error("Missing required parameters: day, venue_id, subject_ids (array), slot.");
}

// Venue not found: logs and silently returns
if (!venueDataRes.length) {
  log(`■ Venue ID ${venue_id} not found.`);
  return;
}
```

Both are "this request can't proceed," but a caller can't distinguish them programmatically — one surfaces as a thrown exception (caught by handleAddtimetable's try/catch), the other only shows up inside the logs array with no exception at all. Fix direction: pick one convention (most likely: always log-and-return within addtimetable, since it already processes subjects independently and reports per-item outcomes — reserve throw only for truly invalid input the caller should have caught before calling this function at all).

Suggested Order to Tackle These

- **Finding 1** first — it is the only one that is an actual bug, not an improvement.
- **Findings 2 and 8** together next — both reduce the same function's query count significantly, and touch the same code path (resolveSlotColumn / venue data).
- **Findings 4, 5, 6, 7** — safe, mechanical cleanups fully contained within this one file.
- **Finding 3** last — the biggest-scope change, since it touches tmasterModel.js and collisionMonitorLogic.js as well, and is worth doing deliberately once the in-file cleanups are settled.
- **Finding 9** — a style decision worth making consciously alongside Finding 1, since fixing the transaction handling is a natural moment to also settle on one error-reporting convention.